Grafici a punti (dotplot) con ggplot

I **grafici a punti (dotplot)** rappresentano una interessante alternativa ai *grafici a scatola con i baffi (boxplot)* [1] nei casi di distribuzioni univariate che includono pochi dati e per le quali non volete perdere il dettaglio dei singoli valori.

Possono inoltre essere sovrapposti sia ai boxplot sia ai *grafici a violino (violin plot)* [2] a testimonianza del fatto che si tratta di viste dei dati diverse ma tra loro integrate.

Per eseguire i due script che seguono è necessario scaricare dal **CRAN** il pacchetto **ggplot2** [3] e il pacchetto **gridExtra** [4]. Accertatevi inoltre di avere già installato il pacchetto **DAAG** che contiene i dati impiegati nello script, o in alternativa procedete seguendo le indicazioni riportate in [5].

Copiate questo primo script, incollatelo nella `Console di R` e premete `↵ Invio`.

```
# UN GRAFICO A PUNTI (DOTPLOT)
#
library(DAAG) # carica il pacchetto DAAG che include il set di dati ais
library(ggplot2) # carica il pacchetto necessario per la grafica
#
ggplot(ais, aes(x=sport, y=hg, fill=sport)) + geom_dotplot(method="dotdensity",
binaxis='y', stackdir="center", stackratio=1, dotsize=1, binwidth=0.2, show.legend=FALSE)
+ coord_cartesian(ylim=c(10, 20)) + labs(title="Concentrazione dell'emoglobina nel
sangue per sport praticato", x="Sport praticato", y="Emoglobina (g/dL)") +
theme_classic() # traccia dotplot dell'emoglobina per sport
#
```

Dopo avere caricato i pacchetti necessari, il grafico a punti (dotplot) viene realizzato impiegando una sola riga di codice che include cinque funzioni concatenate:
→ la funzione **ggplot()** che specifica i dati da impiegare (**ais**) e con la funzione **aes()** specifica di realizzare il grafico a punti separatamente per ciascuno sport praticato (**x=sport**) per la variabile emoglobina (**y=hg**);
→ la funzione **geom_dotplot()** che di fatto realizza il grafico con una serie piuttosto articolata di argomenti per il dettaglio dei quali rimando alla documentazione della funzione inclusa nel manuale di riferimento del pacchetto [3];
→ la funzione **coord_cartesian()** che viene impiegata per stabilire (**ylim=**) valore minimo e valore massimo pare rappresentare la distribuzione dei punti sull'asse delle y;
→ la funzione **labs()** che banalmente riporta titolo ed etichette degli assi;
→ la funzione **theme_classic()** che definisce "il look" del grafico.

In generale per realizzare un grafico a punti (dotplot) è necessario avere una <u>variabile numerica</u> [`valore`] che contiene i valori osservati più una seconda <u>variabile qualitativa</u> che contiene un criterio di classificazione [`fattore`] per aggregare i valori in sottoinsiemi, come qui schematicamente esemplificato

valore	fattore
4	A
2	B
5	B
4	A
3	C
5	A
2	C
7	B
...	...

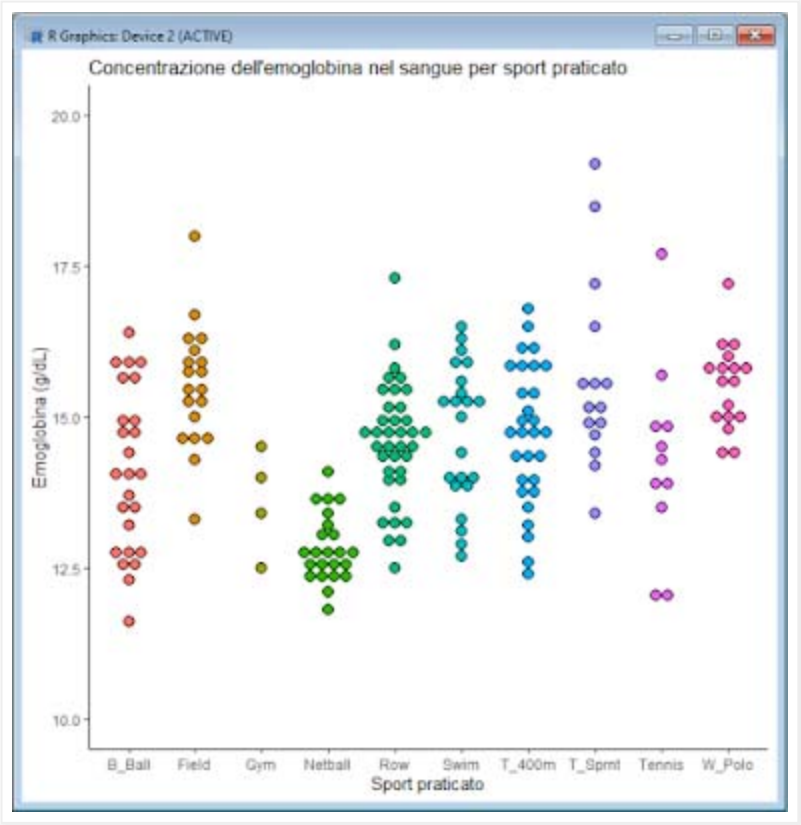
e dove nel caso specifico il `valore` è la concentrazione di emoglobina contenuta nella variabile **hg** (riportata sull'asse verticale **y**) e il `fattore` è lo sport praticato contenuto nella variabile **sport** (riportata sull'asse orizzontale **x**).

Se guardate alla struttura dell'oggetto **ais** digitando

str(ais)

notate che i dati impiegati per realizzare il grafico a punti sono strutturati proprio in questo modo.

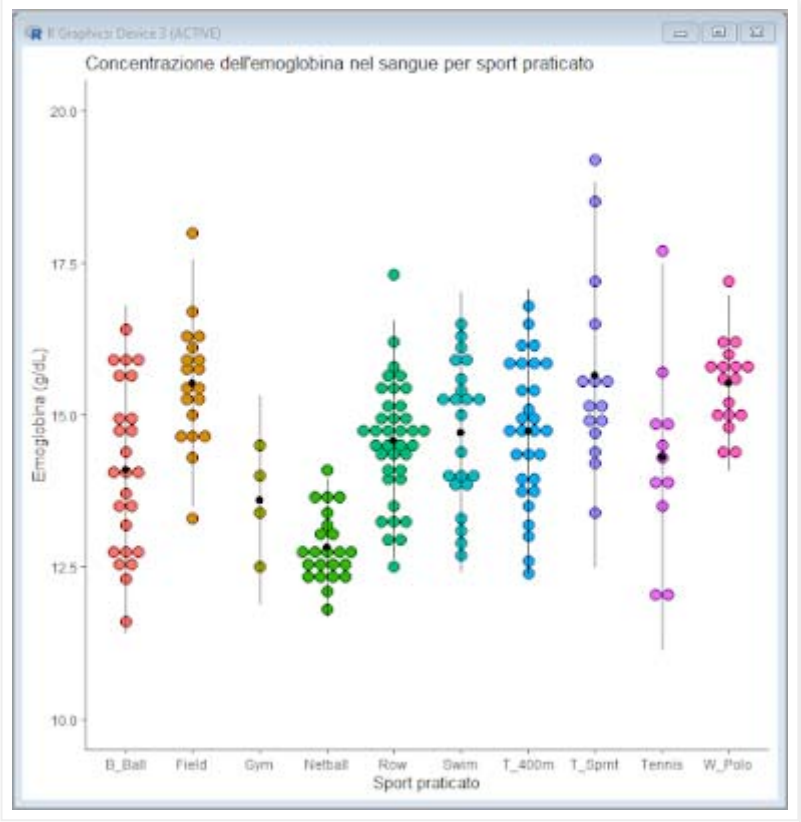
E questo è il grafico ottenuto con lo script.



Ora copiate e incollate nella `Console di R` queste due altre righe di codice che aprono una nuova finestra grafica nella quale viene rappresentato lo stesso grafico, questa volta aggiungendo (**+**) al codice riportato in precedenza la funzione **stat_summary()** che sovrappone ai punti la media e l'intervallo corrispondente a due (**mult=2**) deviazioni standard:

```
#
windows() # apre e inizializza una nuova finestra grafica
ggplot(ais, aes(x=sport, y=hg, fill=sport)) + geom_dotplot(method="dotdensity",
binaxis='y', stackdir="center", stackratio=1, dotsize=1, binwidth=0.2,
show.legend=FALSE) + coord_cartesian(ylim=c(10, 20)) + labs(title="Concentrazione
dell'emoglobina nel sangue per sport praticato", x="Sport praticato", y="Emoglobina
(g/dL)") + theme_classic() + stat_summary(fun.data=mean_sdl, fun.args = list(mult=2),
geom="pointrange", color="black", show.legend=FALSE) # aggiunge media +/- 2 deviazioni
standard
#
```

Questo è il grafico.

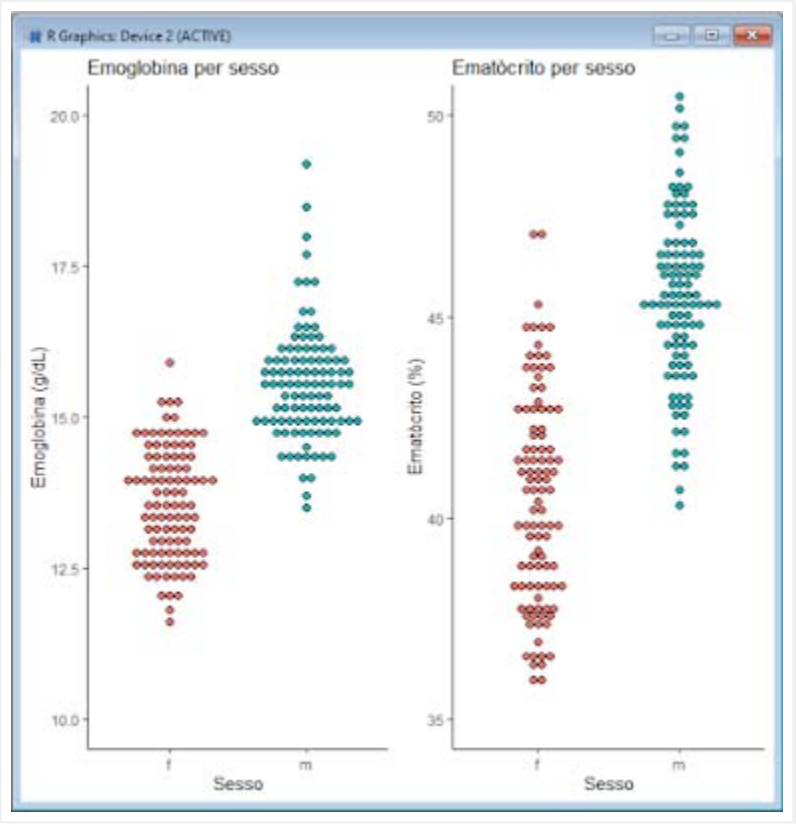


Copiate quest'altro script, incollatelo nella `Console di R` e premete ↵ `Invio`.

DUE GRAFICI A PUNTI (DOTPLOT) AFFIANCATI
#
library(DAAG) # carica il pacchetto DAAG incluso il set di dati ais
library(ggplot2) # carica il pacchetto necessario per la grafica
#
plot1 <- ggplot(ais, aes(x=sex, y=hg, fill=sex)) + geom_dotplot(method="dotdensity", binaxis='y', stackdir="center", stackratio=1, dotsize=0.7, binwidth=0.2, show.legend=FALSE) + coord_cartesian(ylim=c(10, 20)) + labs(title="Emoglobina per sesso", x="Sesso", y="Emoglobina (g/dL)") + theme_classic() # dotplot dell'emoglobina per sesso
#
plot2 <- ggplot(ais, aes(x=sex, y=hc, fill=sex)) + geom_dotplot(method="dotdensity", binaxis='y', stackdir="center", stackratio=1, dotsize=1, binwidth=0.2, show.legend=FALSE) + coord_cartesian(ylim=c(35, 50)) + labs(title="Ematòcrito per sesso", x="Sesso", y="Ematòcrito (%)") + theme_classic() # dotplot dell'ematòcrito per sesso
#
library(gridExtra) # carica il pacchetto con la funzione necessaria per realizzare i grafici affiancati
grid.arrange(plot1, plot2, nrow = 1) # i due grafici sono affiancati orizzontalmente
#

Il codice di fatto è sempre lo stesso tranne che per due aspetti:
→ sono realizzati due grafici a punti - per la variabile **hg** e per la variabile **hc** [5] - separando i dati per sesso (**x=sex**);
→ per affiancare i due grafici, dell'emoglobina (**hg**) per sesso e dell'ematòcrito (**hc**) per sesso, viene impiegata la funzione **grid.arrange()** inclusa nel pacchetto **gridExtra** [3] che consente di superare i limiti della analoga funzione **par(mfrow=...)** del *pacchetto base* di **R** [6].

Questo è il risultato.



Per adattare gli script ai vostri dati dovete:
→ sostituire la prima riga con il codice necessario per importare i vostri dati;
→ nella funzione **ggplot()** sostituire **ais** con il nome dell'oggetto che contiene i vostri dati;
→ nella funzione **aes()** sostituire in **x** il nome della variabile (**sex**) con quello della variabile (fattore) che volete impiegare per raggruppare i vostri valori in sottoinsiemi;
→ nella funzione **aes()** sostituire in **y** il nome della variabile (**hg**) con quello della variabile numerica che contiene i vostri valori;
→ nella funzione **labs()** adattare opportunamente titolo, nome del fattore e nome della variabile.

Ricordate infine che:
→ potete visualizzare il kernel density plot dei dati sovrapponendo ai grafici a punti i grafici a violino [2];
→ potete salvare le immagini realizzate con **R** sotto forma di file `.bmp`, `.jpeg`, `.png`, `.pdf`, `.ps` per stamparle, archiviarle, inserirle in una pubblicazione, in un post o in un sito web, impiegando il codice riportato nel post Salvare i grafici di R in un file.

Se siete interessati a rappresentazioni grafiche più elaborate di quelle presenti nel pacchetto base di **R** potrebbe esservi utile approfondire le funzioni del pacchetto **ggplot2** [3].

---

[1] Vedere il post Grafici a scatola con i baffi (boxplot).

[2] Vedere il post Grafici a violino (violin plot).

[3] Il manuale di riferimento *Package 'ggplot2'* lo trovate facendo click sul link *Reference manual* nella pagina di documentazione del pacchetto.
https://cran.r-project.org/web/packages/ggplot2/index.html

[4] Il manuale di riferimento *Package 'gridExtra'* lo trovate facendo click sul link *Reference manual* nella relativa pagina di documentazione del pacchetto.
https://cran.r-project.org/web/packages/gridExtra/index.html

[5] Vedere il post Il set di dati ais e le indicazioni riportate alla pagina Dati.

[6] Alcuni esempi di impiego della funzione **par(mfrow=...)** sono riportati nel post Inserire più grafici nella stessa immagine.